UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Source: CSC429-2023-07, page [2]

Unofficial model answer:

a) The Von Neumann architecture consists of a CPU (containing the control unit and ALU), memory, input devices, output devices, and secondary storage, all connected by a shared bus system. The control unit fetches and decodes instructions, the ALU performs arithmetic and logic operations, memory holds both instructions and data, input devices supply data (e.g., keyboard, mouse), output devices present results (e.g., monitor, printer), and secondary storage provides non-volatile storage (e.g., hard disk, SSD). The key feature is the stored-program concept where instructions and data reside in the same memory and are accessed via a shared bus, leading to the Von Neumann bottleneck. A diagram should show these components with arrows indicating data/address/control flow.

b) The actual device that John Von Neumann himself developed is the IAS (Institute for Advanced Study) machine, also known as the Von Neumann machine. It was built at the Institute for Advanced Study in Princeton and is considered the first stored-program computer.

Marking points:

• 1 mark: Correctly drawn block diagram showing CPU (control unit and ALU), memory, input, output, and secondary storage connected by buses.

• 1 mark: Relevant example given for each component (e.g., keyboard for input, monitor for output, hard disk for secondary storage, etc.).

• 1 mark: Clear indication of the stored-program concept or shared bus for instructions and data.

• 1 mark: Correctly naming the IAS machine (or Von Neumann machine) as the device developed by John Von Neumann.

• 1 mark: Additional detail such as the location (Institute for Advanced Study) or its significance as the first stored-program computer.

Model confidence: high.

Model note: The question asks to 'draw' the architecture; the answer describes the diagram. The marking scheme assumes a typical block diagram is expected. The device in part (b) is commonly known as the IAS machine; some sources may refer to it as the 'Von Neumann machine' or 'IAS computer'.

QUESTION A2 (5 marks)

Source: CSC429-2017-07, page [3]

Unofficial model answer:

a) The generation described is the Fourth Generation of computers (the microprocessor generation, roughly 1971 onwards). The statement's markers — GUI and mouse technology, plus major advances in computer networking that led to the internet concept — identify this as the fourth generation.

b) The two main technologies available in this generation are: (i) microprocessors / very-large-scale integration (VLSI) integrated circuits, which put the CPU on a single chip; and (ii) graphical user interface (GUI) technology together with the mouse pointing device (supported by networking technology).

c) Two main features of the fourth generation are: (i) use of microprocessors/VLSI chips, giving smaller, cheaper, more reliable and faster machines (e.g. personal computers); and (ii) widespread GUI-based interaction with mouse input, along with the growth of computer networks and the emergence of the internet concept.

Marking points:

• 1 mark (a): Correctly names the Fourth Generation (microprocessor generation).

• 1 mark (b): Identifies microprocessors / VLSI integrated circuits as a main technology.

© Hak Cipta Universiti Teknologi MARA

• 1 mark (b): Identifies GUI and mouse (graphical user interface) technology as a main technology.

• 1 mark (c): Gives one valid feature, e.g. microprocessor/VLSI-based hardware, smaller/cheaper personal computers, or GUI/mouse interaction.

• 1 mark (c): Gives a second valid feature, e.g. growth of computer networks / birth of the internet concept, or high-level/structured programming languages.

Model confidence: high.

Model note: Answer derived from the question statement and standard CSC429 generation classifications; no figure or table was supplied. The paper's own lecture notes may phrase the fourth-generation technologies/features slightly differently, so accept equivalent wording (e.g. 'LSI/VLSI' for microprocessors, 'networking/internet' as a feature). This is an unofficial model answer, not an official UTM marking scheme.

QUESTION A3 (5 marks)

Source: CSC429-2024-01, page [3]

Unofficial model answer:

a) Sign and magnitude method for 138 - 518: First, represent both operands as 8-bit sign-magnitude bytes. +138 = 10001010 (sign bit 0, magnitude 138 = 10001010). +518 cannot be represented in 8-bit sign-magnitude because the magnitude 518 exceeds 7-bit range (max 127). Therefore, the operation 138 - 518 = -380 cannot be performed directly in 8-bit sign-magnitude. To show a workflow, we note that 138 - 518 = -(518 - 138) = -380. In 8-bit sign-magnitude, -380 is also not representable (magnitude 380 > 127). Thus the result overflows the byte format. If we assume a larger format, e.g., 16-bit sign-magnitude: +138 = 0000000010001010, +518 = 0000001000000110. Subtract: 518 - 138 = 380 = 0000000110111100 (since 518-138=380). Then apply negative sign: 1000000110111100. But the question requires byte form, so the correct answer is that the operation cannot be completed in 8-bit sign-magnitude due to overflow; the result -380 is out of range. For marking, state the overflow and show the attempt. b) -23 + 69 using 1's and 2's complement: Represent +69 and -23 in 8-bit. +69 = 01000101. -23: magnitude 23 = 00010111, so 1's complement = 11101000, 2's complement = 11101001. 1's complement addition: 01000101 + 11101000 = 1 00101101 (carry out 1). Add carry to LSB: 00101101 + 1 = 00101110 = +46. 2's complement addition: 01000101 + 11101001 = 1 00101110 (carry out 1, discarded). Result = 00101110 = +46. Both methods give +46, which is correct since -23+69=46. Byte form: 00101110.

Marking points:

• 1 mark: Correctly represent +138 as 10001010 in 8-bit sign-magnitude.

• 1 mark: Recognize that +518 cannot be represented in 8-bit sign-magnitude (magnitude > 127) and thus the operation overflows; state that result -380 is not representable in a byte.

• 1 mark: Show correct 1's complement representation of -23 (11101000) and +69 (01000101).

• 1 mark: Perform 1's complement addition correctly, including end-around carry, to get 00101110 (+46).

• 1 mark: Perform 2's complement addition correctly (discard carry) to get 00101110 (+46) and state final byte answer.

Model confidence: medium.

Model note: Assumption: 'byte form' means 8-bit representation. For part (a), the operands exceed 8-bit sign-magnitude range, so the operation cannot be completed; the answer highlights overflow. If the question intended a larger byte size (e.g., 16-bit), the workflow would differ, but 'byte' typically means 8 bits. Part (b) is straightforward and verified.

QUESTION A4 (5 marks)

Source: CSC429-2016-12, page [2]

Unofficial model answer:

a) Machine X is the Jacquard Loom (the Jacquard weaving machine/attachment). The placard describes Joseph Marie Jacquard's system that used a deck of punched cards to control the weaving of intricate patterns, which is the defining feature of the Jacquard Loom.

b) The Jacquard Loom was developed in 1801, i.e. in the pre-electronic/mechanical era — it belongs to the era

© Hak Cipta Universiti Teknologi MARA

before the first electronic computer generation, and is conventionally associated with the 'zeroth'/mechanical generation (the era of mechanical calculating and punched-card machines, later extended by Hollerith). It is NOT one of the electronic computer generations (first to fifth); it predates them and is regarded as an early punched-card/mechanical-generation device.

c) The deck of punched cards functions as a stored program/instruction store that directs the machine's operations. The analogous computer component is memory (main store/RAM), specifically the program/instruction store — the punched cards hold the sequence of control instructions (a stored program) that the machine reads and executes. (Accept: ROM/program memory, or the control unit reading instructions from memory.)

d) Input: the deck of punched cards (the punched-card pattern program) together with the yarn/thread and the operator's setup. Output: the woven fabric/textile with the intricate pattern produced by the loom.

Marking points:

• 1 mark (a): Correctly names Machine X as the Jacquard Loom (accept Jacquard weaving machine/loom).

• 1 mark (b): States the correct era — developed in 1801, in the mechanical/pre-electronic ('zeroth') generation, before the electronic computer generations.

• 1 mark (c): Identifies memory/program store (RAM/ROM/instruction store) as the component with a similar function to the punched-card deck.

• 1 mark (d) input: Identifies the punched cards (and yarn) as input.

• 1 mark (d) output: Identifies the woven patterned fabric/textile as output.

Model confidence: medium.

Model note: Figure 1 is supplied only by caption/extracted text; the answer relies on the placard text, which clearly identifies the Jacquard system. Part (b) is ambiguous because the Jacquard Loom predates electronic computer generations; the answer gives the defensible 'mechanical/pre-electronic (zeroth) generation, 1801' interpretation. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION A5 (10 marks)

Source: CSC429-2024-07, page [4]

Unofficial model answer:

a) Three types of replacement algorithm: (1) FIFO (First-In-First-Out): the cache line that has been resident in the set the longest is chosen for replacement, maintained by a circular queue or counter; it is simple but can evict a frequently used line (Belady's anomaly possible). (2) LRU (Least Recently Used): the line that has not been referenced for the longest time is replaced, based on temporal locality; it gives good hit ratios but requires tracking recency (e.g., counters or a stack) and is costly for high associativity. (3) Random: a pseudo-randomly selected line in the set is evicted; it needs minimal hardware, avoids pathological patterns, and performs nearly as well as LRU for high associativity. (Other acceptable: LFU, NRU, optimal/OPT.)

b) Two mapping types that require a replacement algorithm: (1) Set-associative mapping (each main-memory block maps to a set of lines, so a choice within the set is needed); (2) Fully associative mapping (a block may occupy any cache line, so a replacement policy is always needed). Direct mapping does not require replacement because each block has exactly one possible line.

c) Reason for implementing a replacement algorithm: when a cache miss occurs and the target set (or the whole cache in fully associative mapping) is full, an existing valid line must be evicted to make room for the new block. The replacement algorithm decides which line to remove in order to maximise future hits by exploiting locality (e.g., keeping recently used lines). A good policy reduces the miss rate and thus the average memory access time (AMAT = hit time + miss rate × miss penalty), improving overall system performance; without a policy, the choice would be arbitrary and could cause excessive thrashing.

Marking points:

• 1 mark: correct description of FIFO replacement (oldest line evicted).

• 1 mark: correct description of LRU replacement (least recently used line evicted).

© Hak Cipta Universiti Teknologi MARA

• 1 mark: correct description of a third algorithm such as Random (or LFU/NRU/OPT) with its basis.

• 1 mark: set-associative mapping listed as requiring replacement.

• 1 mark: fully associative mapping listed as requiring replacement.

• 1 mark: explanation that replacement is needed when a cache miss occurs and the target set/cache is full.

• 1 mark: explanation that a line must be evicted to load the new block.

• 1 mark: link to exploiting locality / keeping frequently or recently used data.

• 1 mark: link to reducing miss rate and improving average memory access time/performance.

• 1 mark: overall coherence and correct terminology (e.g., eviction, set, valid line, AMAT).

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure or table was supplied, so the answer is based solely on the question text. Part (a) asks for three types; FIFO, LRU and Random are given as the most common examples, but other valid algorithms (LFU, NRU, OPT) would also earn credit.

QUESTION A6 (10 marks)

Source: CSC429-2015-12, page [6]

Unofficial model answer:

a) Examples for each classification of multiple processor organization: (i) Symmetric Multiprocessors (SMP): a dual-socket Intel Xeon server board with two identical CPUs sharing a single main memory and I/O subsystem through a common system bus or crossbar; (ii) Clusters: a Beowulf cluster of commodity PCs connected by an Ethernet/Gigabit network, each node running its own OS and cooperating as a single system; (iii) Non-Uniform Memory Access (NUMA): a modern multi-socket server (e.g., AMD EPYC or Intel Xeon Scalable) where each processor has local memory and accesses remote memory attached to other processors with higher latency. (Other valid examples: SMP = multicore processor with shared L3; Clusters = Hadoop cluster; NUMA = SGI Origin 2000.)

b) Simplified architectures (described textually because diagrams cannot be drawn here):

i. Symmetric Multiprocessors (SMP): A single shared main memory is connected via a system bus (or crossbar switch) to multiple identical processors (P1, P2, ..., Pn). Each processor also connects to the same bus for I/O devices. All processors have equal access to all memory and I/O, and a single OS manages the whole system. Diagram: [P1][P2]...[Pn] each attached to a common bus; bus attached to shared memory and I/O.

ii. Clusters: Multiple independent computer nodes (each node = processor + local memory + local I/O + OS) interconnected by a high-speed network (e.g., Ethernet, InfiniBand) through network interfaces. No shared global memory; communication via message passing. Diagram: several node boxes, each containing CPU, memory, I/O, connected to a central network switch/hub.

iii. NUMA: Multiple processors (or processor groups) each with its own local memory, interconnected by a scalable interconnection network (e.g., crossbar or mesh). A processor accesses its local memory directly (low latency) and remote memory through the interconnection network (higher latency). I/O devices may be attached to some nodes. Diagram: [P1 + local memory] --- interconnection network --- [P2 + local memory] --- ... with I/O attached to nodes.

Marking points:

• 1 mark: Correct example for Symmetric Multiprocessors (e.g., dual-socket server with shared memory).

• 1 mark: Correct example for Clusters (e.g., Beowulf cluster of PCs with network).

• 1 mark: Correct example for NUMA (e.g., multi-socket server with local/remote memory).

• 1 mark: SMP diagram shows multiple identical processors connected to a common bus/shared memory.

• 1 mark: SMP diagram indicates equal access to shared memory and I/O.

• 1 mark: Clusters diagram shows multiple independent nodes (each with CPU, memory, I/O) connected via a network.

• 1 mark: Clusters diagram indicates no shared memory (message passing).

• 1 mark: NUMA diagram shows processors with local memory and an interconnection network.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: NUMA diagram distinguishes local vs. remote memory access (different latency).

• 1 mark: Overall clarity and correct labeling of components in all three diagrams.

Model confidence: high.

Model note: The question asks for drawings; since diagrams cannot be produced in this text format, the answer describes the required architectures in words. The marking scheme assumes the examiner awards marks for the presence and correctness of key components in each diagram. Examples given are illustrative; other valid examples should be accepted.

QUESTION A7 (10 marks)

Source: CSC429-2018-01, page [5]

Unofficial model answer:

a) The addressing mode in Figure 2 is **register indirect addressing**. The instruction specifies a register (in the register field) that holds the effective address of the operand, rather than the operand itself. The operand is fetched from memory at the address contained in that register.

b) The register involved is a **general-purpose register** (e.g., R1, R2, etc.) that contains the memory address of the operand. The specific register number is given in the instruction's register field. In many architectures, any general-purpose register can be used for register indirect addressing.

c) If the register in Figure 2 is replaced with a memory location, the new addressing mode is **memory indirect addressing** (also called indirect addressing). In this mode, the instruction specifies a memory address (via the address field) that contains the effective address of the operand. Thus, the operand is accessed through two memory references: one to fetch the effective address and one to fetch the operand.

d) **Register indirect addressing executes faster** than memory indirect addressing. Justification: Register indirect addressing requires only one memory access to fetch the operand (after the effective address is already in the register). Memory indirect addressing requires at least two memory accesses: one to read the effective address from memory and another to read the operand. Since memory accesses are much slower than register accesses, register indirect addressing is significantly faster. Additionally, registers are located within the CPU, so accessing them incurs no memory latency, whereas memory indirect involves the slower main memory system.

Marking points:

• 1 mark: (a) Correctly identifies register indirect addressing.

• 1 mark: (b) States that a general-purpose register is involved (or names a specific register if shown in figure).

• 1 mark: (c) Correctly identifies memory indirect addressing (or indirect addressing).

• 2 marks: (d) States register indirect is faster (1 mark) and provides justification: fewer memory accesses (1 mark).

• 2 marks: (d) Additional justification: registers are faster than memory (1 mark) and clear comparison of access counts (1 mark).

• 3 marks: Overall clarity, correct terminology, and completeness of discussion in (d).

Model confidence: medium.

Model note: Figure 2 is described only by its caption; the actual diagram is not available. The answer assumes the figure shows an instruction with an opcode, address field, register field, and operand, typical of register indirect addressing. If the figure depicts a different format, the answer may need adjustment. The marking scheme is unofficial and based on typical CSC429 expectations.

QUESTION A8 (10 marks)

Source: CSC429-2023-07, page [3]

Unofficial model answer:

a) The machine cycle (also called the instruction cycle) consists of the phases: Fetch, Decode, Execute, and (where applicable) Memory access and Write-back. In a simple model, the phases are: instruction fetch (IF), instruction decode (ID), operand fetch / memory access (MEM), execute (EX), and write-back (WB). Some textbooks combine operand fetch with execute, but the standard CSC429 list is: Fetch, Decode, Execute, and Store/Write-back.

b) The CPU components and their functions in the machine cycle: The Program Counter (PC) holds the address of the next instruction; during fetch, the PC content is placed on the address bus and incremented. The Memory Address Register (MAR) holds the address of the memory location to be accessed (instruction or operand). The Memory Data Register (MDR) / Memory Buffer Register (MBR) holds the data fetched from or to be written to memory. The Instruction Register (IR) holds the current instruction during decode and execute. The Control Unit (CU) decodes the instruction in the IR and generates the control signals that coordinate all other components (e.g., read/write memory, select ALU operation, enable registers). The Arithmetic Logic Unit (ALU) performs arithmetic and logical operations during execute. General-purpose registers hold operands and results. The accumulator (if present) may hold one operand and the result. The status/flags register records conditions (zero, carry, etc.) from ALU operations. Buses (address, data, control) transfer addresses, data, and control signals between CPU and memory/I/O.

c) If access to main memory is blocked (e.g., memory failure, bus contention, or a memory management fault), the machine cycle cannot complete the fetch phase or any memory access phase. The CPU will stall or wait, because it cannot fetch the next instruction or read/write operands. Typically, the control unit will detect a memory access fault (e.g., bus error, page fault, or timeout) and may generate an exception or interrupt. The processor may enter a halt state or invoke an exception handler. Without memory access, no new instructions can be fetched, so the program cannot proceed; the system may hang or require a reset. If the block is temporary (e.g., wait states), the CPU inserts wait states and the cycle is stretched but eventually completes when memory becomes available.

Marking points:

• *1 mark: Correctly listing the phases of the machine cycle (Fetch, Decode, Execute, and optionally Memory/Write-back).*

• *1 mark: Naming at least four CPU components (PC, MAR, MDR, IR, CU, ALU, registers).*

• *1 mark: Describing the role of PC and MAR in fetch.*

• *1 mark: Describing the role of MDR/MBR and IR in fetch/decode.*

• *1 mark: Describing the role of the Control Unit in decode and control signal generation.*

• *1 mark: Describing the role of ALU and registers in execute.*

• *1 mark: Explaining that blocked memory access prevents instruction fetch and operand access.*

• *1 mark: Explaining that the CPU stalls or waits (wait states) or generates an exception/interrupt.*

• *1 mark: Explaining the consequence: program cannot proceed, system may halt or invoke handler.*

• *1 mark: Overall clarity and correct use of terminology.*

Model confidence: high.

Model note: The question does not specify a particular CPU model; the answer assumes a standard von Neumann architecture with a simple instruction cycle. Some textbooks include 'operand fetch' as a separate phase; the answer notes this variation. No figure or table was provided.

QUESTION A9 (10 marks)

Source: CSC429-2016-06, page [3]

Unofficial model answer:

a) Decimal to 6-bit binary: For 13: repeatedly divide by 2: 13/2=6 remainder 1 (LSB), 6/2=3 remainder 0, 3/2=1 remainder 1, 1/2=0 remainder 1 (MSB). Reading remainders upward gives 1101. Pad to 6 bits: 001101. For 31: 31/2=15 r1, 15/2=7 r1, 7/2=3 r1, 3/2=1 r1, 1/2=0 r1. Reading upward gives 11111. Pad to 6 bits: 011111. Thus 13 = 001101_2 and 31 = 011111_2 in 6-bit representation.

b) Sign-and-magnitude representation uses the most significant bit as the sign bit (0 for positive, 1 for negative) and the remaining 5 bits as the magnitude. In 6 bits, +13 = 0 01101 and -13 = 1 01101; +31 = 0 11111 and -31 = 1 11111. Addition/subtraction is performed by converting subtraction to addition of the negated operand, then applying sign-and-magnitude addition rules: if signs are the same, add magnitudes and keep the sign; if signs differ, subtract the smaller magnitude from the larger and take the sign of the larger magnitude. Overflow is possible if the magnitude sum exceeds 31 (5 bits).

© Hak Cipta Universiti Teknologi MARA

i) 13 + 31: Both positive. Magnitudes: 01101_2 (13) + 11111_2 (31) = 01101 + 11111 = 101100_2, which is 44 in decimal. Since 44 > 31, the 5-bit magnitude field overflows. In 6-bit sign-and-magnitude, the result cannot be represented correctly; the magnitude bits would be 01100 (12) with a carry out, giving an incorrect result if truncated. Correct mathematical result is +44, which requires at least 7 bits in sign-and-magnitude (0 101100). So in 6-bit sign-and-magnitude, 13+31 causes overflow.

ii) 13 - 31 = 13 + (-31). Signs differ. Magnitudes: |13| = 01101, |31| = 11111. Subtract smaller from larger: 11111 - 01101 = 10010_2 (18). Sign of larger magnitude is negative (31 is negative). Result: 1 10010_2 = -18. No overflow since 18 ≤ 31.

iii) -13 + 31 = (-13) + 31. Signs differ. Magnitudes: |−13| = 01101, |31| = 11111. Subtract smaller from larger: 11111 - 01101 = 10010_2 (18). Sign of larger magnitude is positive (31 is positive). Result: 0 10010_2 = +18. No overflow.

iv) -13 - 31 = (-13) + (-31). Both negative. Magnitudes: 01101 + 11111 = 101100_2 (44). Since 44 > 31, the 5-bit magnitude overflows. In 6-bit sign-and-magnitude, the result cannot be represented correctly; truncated magnitude would be 01100 (12) with sign 1, giving -12, which is wrong. Correct mathematical result is -44, requiring at least 7 bits (1 101100). So in 6-bit sign-and-magnitude, -13-31 causes overflow.

Marking points:

• *1 mark: Correct conversion of 13 to 6-bit binary 001101 with division workflow shown.*

• *1 mark: Correct conversion of 31 to 6-bit binary 011111 with division workflow shown.*

• *1 mark: Correct sign-and-magnitude representation of +13, -13, +31, -31 in 6 bits (0 01101, 1 01101, 0 11111, 1 11111).*

• *1 mark: Correct method for i) 13+31: add magnitudes, detect overflow because 44 > 31; state result not representable in 6-bit sign-and-magnitude.*

• *1 mark: Correct result for ii) 13-31 = -18 (1 10010) with working showing subtraction of magnitudes and sign selection.*

• *1 mark: Correct result for iii) -13+31 = +18 (0 10010) with working showing subtraction of magnitudes and sign selection.*

• *1 mark: Correct method for iv) -13-31: add magnitudes, detect overflow because 44 > 31; state result not representable in 6-bit sign-and-magnitude.*

• *1 mark: Clear workflow for each equation, showing conversion to sign-and-magnitude, operation on magnitudes, sign determination, and overflow check.*

• *1 mark: Correct final answers for all four parts (including overflow statements for i and iv).*

• *1 mark: Overall accuracy and completeness, including correct 6-bit formatting and no arithmetic errors.*

Model confidence: high.

Model note: Assumes standard sign-and-magnitude representation with 1 sign bit and 5 magnitude bits (6 bits total). Overflow occurs when the true sum magnitude exceeds 31. The question asks to 'solve' using sign-and-magnitude; for cases where the result exceeds 6-bit range, the correct answer is to indicate overflow rather than give a truncated value. No figure or table was provided.

QUESTION A10 (10 marks)

Source: CSC429-2024-01, page [4]

Unofficial model answer:

a) Two methods to handle multiple interrupts: (1) Priority-based (vectored) interrupt handling — each interrupt source is assigned a priority, and when several interrupts arrive simultaneously the CPU services the highest-priority one first; lower-priority requests are held pending and serviced later, often using an interrupt priority encoder or daisy-chain arbitration. (2) Interrupt masking/disable — the processor can selectively mask (disable) certain interrupts while servicing a critical section or a higher-priority interrupt, so lower-priority sources cannot preempt; the masked requests remain pending until unmasked. (Nested interrupt handling, where a higher-priority interrupt can preempt the current ISR, is a common variant of the priority method.)

© Hak Cipta Universiti Teknologi MARA

b) One example of a non-maskable interrupt (NMI): a power failure / power-supply voltage drop warning, or a memory parity/ECC error (also hardware fault or reset). Any one of these is acceptable; the key is that it cannot be disabled by software and signals a catastrophic condition.

c) DMA is faster than interrupt-driven and programmed I/O because the DMA controller transfers blocks of data directly between the I/O device and main memory without the CPU executing an instruction for each word. In programmed I/O the CPU busy-waits and moves every word itself; in interrupt-driven I/O the CPU is interrupted for every word/byte and must run an ISR (context save/restore, instruction fetch) per transfer. DMA performs the whole block transfer in hardware, interrupting the CPU only once at completion (or per block), so it removes per-word CPU overhead and instruction-fetch/ISR overhead, giving much higher throughput.

d) A 'hot connection' means the USB device can be connected (plugged in) and disconnected (unplugged) while the system is powered on and running, without shutting down or rebooting the computer; the interface is designed to be safely hot-swappable and the OS detects and configures the device dynamically.

Marking points:

- *a) 1 mark: correct identification of priority-based/vectored handling with highest-priority-first servicing*
- *a) 1 mark: correct explanation of interrupt masking/disable (or nested/preemption) as a second method*
- *b) 1 mark: valid NMI example (power failure, memory parity/ECC error, hardware fault/reset)*
- *c) 1 mark: DMA transfers data directly between device and memory without CPU per-word involvement*
- *c) 1 mark: contrast with programmed I/O (CPU busy-wait/instruction per word) and interrupt-driven I/O (interrupt/ISR per word)*
- *c) 1 mark: CPU interrupted only once per block, removing per-word overhead and increasing throughput*
- *d) 1 mark: device can be connected/disconnected while system is powered on*
- *d) 1 mark: no shutdown/reboot needed; OS detects/configures dynamically (hot-swappable)*

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied, so the answer relies only on the question text. Mark allocation is indicative and totals 10 marks.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Source: CSC429-2025-02, page [6]

Unofficial model answer:

The equation in Figure 2 is interpreted as y = A - (2C * D) / (B + A^2). Registers A, B, C, D are fixed; x, y, z are temporary; final result must be in y. Table 1 provides one-address (LOADp, STORp, ADDp, SUBp, MULp, DIVp), two-address (MOV(p,q), ADD(p,q), SUB(p,q), MUL(p,q), DIV(p,q)), and three-address (ADD(p,q,r), SUB(p,q,r), MUL(p,q,r), DIV(p,q,r)) instructions. In two- and three-address notations, the first operand is the destination. For one-address, the accumulator is implicit; we use x as accumulator and y as final storage.

One-address program: LOAD A; MUL A; STOR z; LOAD B; ADD z; STOR z; LOAD C; MUL C; MUL D; STOR x; LOAD A; SUB x; STOR y; LOAD x; DIV z; STOR y. Explanation: compute A^2 into z, then B+A^2 into z; compute 2C*D into x; compute A - x into y; finally divide by z and store in y. This uses only the listed one-address instructions and temporary registers x, y, z.

Two-address program: MOV(x, A); MUL(x, A); ADD(x, B); MOV(z, C); MUL(z, C); MUL(z, D); MOV(y, A); SUB(y, z); DIV(y, x). Explanation: x = A^2 + B; z = 2C*D; y = A - z; y = y / x. The final result is in y. All instructions are from the two-address column.

Three-address program: MUL(x, A, A); ADD(x, x, B); MUL(z, C, C); MUL(z, z, D); SUB(y, A, z); DIV(y, y, x). Explanation: x = A^2 + B; z = 2C*D; y = A - z; y = y / x. The final result is in y. All instructions are from the three-address column.

Note: The figure was supplied only as an image asset; the equation is inferred from the extracted text 'y = A - 2C * D / (B + A2)'. If the intended grouping differs (e.g., (2C*D)/(B+A^2) versus 2C*(D/(B+A^2))), the instruction sequences would change accordingly. The answer assumes standard operator precedence and left-to-right evaluation for multiplication/division.

Marking points:

*• 1 mark: Correctly interpret the equation as y = A - (2C * D) / (B + A^2) or equivalent, showing understanding of operator precedence.*

• 1 mark: State that one-address instructions use an implicit accumulator and that x, y, z are used as temporary/final storage.

• 2 marks: One-address program correctly computes A^2 and B+A^2 using LOAD, MUL, ADD, STOR (partial credit for minor sequencing errors).

*• 2 marks: One-address program correctly computes 2C*D using LOAD, MUL, MUL, STOR.*

*• 2 marks: One-address program correctly computes A - (2C*D) and divides by (B+A^2), storing final result in y.*

• 1 mark: Two-address program correctly computes x = A^2 + B using MOV, MUL, ADD.

*• 1 mark: Two-address program correctly computes z = 2C*D using MOV, MUL, MUL.*

• 2 marks: Two-address program correctly computes y = A - z and y = y / x, final result in y.

• 1 mark: Three-address program correctly computes x = A^2 + B using MUL, ADD.

*• 1 mark: Three-address program correctly computes z = 2C*D using MUL, MUL.*

• 2 marks: Three-address program correctly computes y = A - z and y = y / x, final result in y.

• 1 mark: All instructions used are from Table 1 and correct assembly notation (operand order, register names) is followed.

• 1 mark: Overall clarity, correct use of temporary registers, and final result stored in y for all three programs.

• 1 mark: Acknowledges any ambiguity in the figure/equation (e.g., grouping) and states assumptions.

Model confidence: medium.

*Model note: The equation was inferred from the extracted text 'y = A - 2C * D / (B + A2)' because the figure was provided only as an image asset without a machine-readable equation. The answer assumes standard precedence: multiplication and division before subtraction, and left-to-right for * and /. If the intended equation is different (e.g., y = A - 2C * (D/(B+A^2))), the programs would need adjustment. Table 1 was provided as a native table and is used as-is. This is an unofficial model answer, not an official UTM*

© Hak Cipta Universiti Teknologi MARA

marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA